

Atividade 09, Comandos RELATÓRIO

O que foi implementado e resultados, Compiladores, UFPB, julho de 2026

Grupo: Daniel Victor, Tiago Trindade, Yann Fabber, Ralf Dwerich

1. RESUMO DA ENTREGA

Entregamos um compilador completo para a linguagem **Cmd**, escrito em Python 3 puro, que traduz o fonte para assembly x86_64 em sintaxe AT&T e monta o executável chamando o gcc. O pacote inclui as quatro fases do compilador, um interpretador de referência usado como oráculo nos testes, 16 programas de teste válidos, 7 casos de erro e a documentação de uso.

Resultado: 23 de 23 testes passando, cada programa válido conferido nas duas vias de execução. Cobre integralmente a linguagem do guia mais cinco extensões da seção Variações. Nenhuma dependência externa além de Python 3 e gcc.

2. ARQUITETURA FINAL

Fase	Arquivo	Entrada	Saída
Léxica	cmdlang/lexico.py	texto do fonte	lista de tokens com linha e coluna
Sintática	cmdlang/sintatico.py	tokens	árvore sintática abstrata
Semântica	cmdlang/semantico.py	árvore	tabela de símbolos validada
Geração	cmdlang/codigo.py	árvore e tabela	assembly x86_64
Apoio	cmdlang/interp.py	árvore	valor do resultado, usado nos testes
Interface	cmdc.py	linha de comando	tokens, árvore, assembly ou executável

3. O QUE FOI FEITO EM CADA FASE

3.1 Análise léxica

- Novos tokens: {, }, <, >, == e as palavras chave if, else, while, return.
- Ambiguidade do = resolvida testando o par de caracteres antes dos operadores de um caractere. A mesma regra cobre <=, >= e !=.
- Palavras chave reconhecidas como identificador e depois reclassificadas por consulta a um conjunto, como sugere o guia.
- Todo token carrega linha e coluna, o que dá mensagem de erro precisa nas fases seguintes.

3.2 Análise sintática

- Descida recursiva com uma função por não terminal. A lista de declarações termina quando aparece {; a lista de comandos termina quando aparece return ou }.
- Um comando é decidido por um único token de lookahead: if, while, read ou identificador.
- Foi adicionado o nível de precedência das comparações, abaixo de soma e subtração, no mesmo formato das funções aditiva e multiplicativa.
- return não é comando: fica fora de lista_comandos, então usá-lo dentro de um if é erro sintático, conforme o guia. Existe teste para isso.

3.3 Análise semântica

- Tabela de símbolos preserva a ordem de declaração, que é a ordem usada depois na seção BSS.
- Na atribuição verificamos os dois lados: nenhuma variável do lado direito pode ser desconhecida e o alvo do lado esquerdo precisa já estar declarado. Nada é inserido na tabela.
- A expressão de uma declaração só enxerga o que foi declarado antes dela.
- Declaração duplicada é erro. A mensagem informa fase, linha, coluna e o nome da variável.

3.4 Geração de código

- Cada variável vira um símbolo de 8 bytes na seção .bss, lido e escrito de forma relativa a RIP.
- Toda expressão deixa o resultado em RAX. Operação binária avalia o lado direito, empilha, avalia o lado esquerdo, desempilha em RBX e aplica a instrução. Divisão usa cqto antes de idivq.
- Comparações seguem o modelo do guia: xorq em RCX, cmpq, setz, setl ou setg em CL e transferência de RCX para RAX.
- Condicional e repetição usam cmpq \$0, %rax, jz e rótulos numerados por um contador global, garantindo nomes únicos em construções aninhadas.
- A atribuição reaproveita exatamente o código da declaração, já que a verificação semântica passou.

if E { C1 } else { C2 }	while E { C }
<codigo_E>	Inicio0:
cmpq \$0, %rax	<codigo E>
jz Lfalso0	cmpq \$0, %rax
<codigo_C1>	jz Lfim0
jmp Lfim0	<codigo C>
Lfalso0:	jmp Inicio0
<codigo_C2>	Lfim0:
Lfim0:	

4. EXTENSÕES IMPLEMENTADAS

Extensão	Como foi feita
Comparações <= >= !=	Mesmo nível de precedência das outras comparações, geradas com setle, setge e setnz.
Booleanos and or not	Três novos níveis de precedência abaixo das comparações. Avaliação com curto circuito por salto condicional, resultado normalizado em 0 ou 1.
if sem else	Braço opcional na gramática. Não há ambiguidade de dangling else porque as chaves são obrigatórias.
Comando read v;	Lê um inteiro da entrada padrão via scanf e grava direto no símbolo da variável, que precisa estar declarada.
Comentários com #	Descartados no analisador léxico. Usados também para declarar o resultado esperado de cada teste.

5. DIFERENÇAS EM RELAÇÃO AO GUIA

- O resultado é impresso em stdout com `printf`, não devolvido como código de saída. O processo sempre termina com 0.
- Na gramática do guia o fecho `*` sobre o grupo de operadores de comparação é erro de digitação. Aqui cada comparação consome um operador, com associatividade à esquerda.
- Declaração duplicada é tratada como erro semântico, ponto que o guia não define.
- Aritmética de 64 bits com sinal e divisão truncando em direção a zero, seguindo a semântica de `IDIV`. O interpretador de referência replica esse comportamento, inclusive o transbordo.
- Identificadores aceitam `_` e dígitos após a primeira letra.

6. RESULTADOS DOS TESTES

`python3 tests/rodar_testes.py` compila e executa tudo. Cada programa válido roda no interpretador e no binário nativo, e os dois valores são comparados com o esperado declarado no topo do arquivo. Cada programa de erro precisa falhar exatamente na fase indicada. Resultado: **23 de 23 testes passando**, sem divergência entre as duas vias.

Programa válido	Obtido	Caso de erro	Fase obtida
01 minimo	42	e01 var não declarada	semântico
02 delta	8	e02 atrib não declarada	semântico
03 soma	45	e03 declaração duplicada	semântico
04 resto	2	e04 return dentro do if	sintático
05 mdc	6	e05 falta chave no while	sintático
06 comparacoes	47	e06 caractere inválido	léxico
07 logicos	7	e07 falta return	sintático
08 if sem else	30	Os quatro programas do guia (delta, soma, resto e mdc) entraram sem alteração de código, o que confirma compatibilidade com a sintaxe concreta original. Os demais cobrem aninhamento, precedência, divisão com sinal, blocos vazios, entrada e cada extensão.	
09 fatorial	3628800		
10 fibonacci	6765		
11 primos	10		
12 leitura	55		
13 precedencia	11		
14 divisao	3		
15 aninhado	75		
16 blocos vazios	5		

7. COMO EXECUTAR

<code>python3 cmdc.py check</code>	<code>prog.cmd</code>	tres analises, reporta erros
<code>python3 cmdc.py tokens</code>	<code>prog.cmd</code>	lista de tokens
<code>python3 cmdc.py ast</code>	<code>prog.cmd</code>	arvore sintatica
<code>python3 cmdc.py asm</code>	<code>prog.cmd</code>	assembly x86_64 em stdout
<code>python3 cmdc.py build</code>	<code>prog.cmd</code>	executavel via gcc
<code>python3 cmdc.py run</code>	<code>prog.cmd</code>	compila e executa
<code>python3 cmdc.py interp</code>	<code>prog.cmd</code>	interpretador de referencia
<code>make test</code>	roda os 23 testes	
<code>make exemplos</code>	resultado de todos os programas de exemplo	

Requer Python 3.8 ou superior e gcc. Os binários gerados são Linux x86_64. Em outra arquitetura, use `interp`; o próprio runner de testes detecta a plataforma e avisa quando pula a verificação nativa.

8. ADERÊNCIA AO PLANEJAMENTO

Item planejado	Situação	Observação
Quatro fases do compilador	feito	Sem desvio em relação ao plano
Interpretador como oráculo	feito	Pegou de fato divergências durante o desenvolvimento das comparações
Quatro extensões da seção Variações	feito	Entrou também comentário de linha, que não estava no plano e saiu de graça no léxico
Atribuição criando variável nova	não feito	Descartado ainda no planejamento, por conflitar com a verificação semântica pedida na atividade
Cobertura mínima de testes	superado	Plano previa cobrir cada construção, terminamos com 16 programas e 7 casos de erro

9. LIMITAÇÕES E TRABALHO FUTURO

- Sem otimização. Toda expressão passa pela pilha, mesmo quando um registrador bastaria.
- Divisão por zero não é detectada em tempo de compilação e gera SIGFPE no binário. O interpretador reporta erro claro.
- Não há detecção de transbordo aritmético, o comportamento é o do registrador de 64 bits.
- Próximo passo natural, se a atividade seguinte pedir: permitir que a atribuição crie variáveis, inserindo o símbolo na BSS durante a geração.